Lecture 05

Software Re-Engineering

Engr. Maqsood Khatoon

Department of Software Engineering

National University of Computer & Emerging Sciences

maqsood.khatoon@nu.edu.pk

Lecture 05

Reverse Engineering Techniques (Re-documentation, Design Recovery)

Objectives of Reverse Engineering: Reducing Costs, Security Analysis, Integration and Customization, Recovering Lost Source Code, Fixing bugs and maintenance

Reverse Engineering Goals: Coping with Complexity, Recover lost information, Detect Side Effects, Facilitate Reuse

Reverse Engineering Techniques

Re-documentation: Creating a semantically equivalent representation of the system at the same level of abstraction (e.g., generating new technical manuals from code).

- In Simple Terms: It is like creating a new, clearer map of a city that is at the same zoom level as the old, messy map. It describes *what* is there.
- Goal: To provide alternate views (e.g., data flow diagrams, control flow graphs) to help a human audience understand the system's structure .
- Key Characteristic: It is the simplest and oldest form of reverse engineering. It recovers documentation, not lost design insights

Design Recovery: Extracting meaningful higher-level abstractions using code, existing documentation, and personal domain knowledge.

- In Simple Terms: It's like looking at the city map and inferring the city's zoning laws, architectural styles, and the original architect's intent.
- Key Characteristic: It goes beyond *what* the system does to understand *why* it does it and what *concepts* it implements

Re-Documentation

- What it produces:
  - Automatically generated code comments.
  - Dataflow diagrams and data structure charts .
  - Control flow graphs.
  - Call graphs and dependency graphs.
- Purpose:
  - To make implicit knowledge (the code structure) explicit in documentation .
  - To support program understanding for new team members.
  - The "less is more" principle applies—only document what is valuable .

"We don't want to spend money on new computers, so we'll need you to keep the 10,000 old computers we have working."

Design recovery

- The "Concept Recovery Problem": The great challenge is recovering design information from legacy code .
- How it works: It recreates design abstractions from a combination of :
  - Code (the primary source).
  - Existing Documentation (if available, often outdated).
  - Personal Experience (the analyst's expertise).
  - Domain Knowledge (understanding of the business problem, e.g., banking, healthcare).
-

Design recovery

Feature	Re-documentation	Design Recovery
Primary Goal	Create alternate views at the same level	Identify higher-level abstractions
Abstraction Level	Same as source code	Higher than source code
Knowledge Source	Primarily the code itself	Code + Domain Knowledge + Experience
Output Example	Control flow graph, call graph	Architecture pattern, business rule
Question Answered	*What* is the structure?	*Why* is it built this way? *What* is the intent?

Reverse Engineering is essential for the survival and evolution of legacy software systems.

Re-documentation helps us see the system clearly by providing new, useful views of its existing structure.

Design Recovery helps us understand the "soul" of the system by uncovering the designer's intent and mapping code to real-world concepts.

Modern techniques leverage AI (constraint satisfaction, ontologies) and structured models (5W1H) to make these processes more effective .

Ultimately, these techniques reduce the effort, cost, and risk of maintaining and modernizing the software that runs our world.

Objectives of Reverse Engineering

- Reducing Costs
- Security Analysis
- Integration and Customization
- Recovering Lost Source Code
- Fixing bugs and maintenance

Reducing Cost

Objective: Cut costs in product development by finding replacements or cost-effective alternatives for systems or components.

Application:

- Identifying redundant code that can be eliminated
- Discovering opportunities for component reuse
- Avoiding complete system rewrites by understanding existing investments

Security Analysis

Objective: Examine exploits, vulnerabilities, and malware to understand threat mechanisms and develop practical defenses.

Application:

- Security researchers dissect malicious binaries to determine behavior, infection vectors, and exploit mechanisms
- Identifying indicators of compromise (IoCs) and hidden payloads
- Understanding adversarial tactics, techniques, and procedures (TTPs)

Integration and Customization

Objective: Incorporate or modify components into pre-existing systems to improve operation or tailor them to meet particular needs.

Application:

- Analyzing protocols, data formats, and APIs to build adapters or bridges
- Enabling communication between proprietary or undocumented systems
- Integrating third-party services with new platforms

Recovering Lost Source Code

Objective:Recover source code that has been lost or is inaccessible, or produce higher-level representations of it.

Application:

- When original source code is unavailable (bankruptcy of vendor, natural disaster)
- Legacy systems where only executables remain
- Producing at least a higher-level understanding of system functionality

Fixing Bugs and Maintenance

Objective: Find and repair flaws or provide updates for systems where original source code is unavailable or inadequately documented.

Application:

- Identifying architectural flaws, redundant code, or hidden dependencies
- Revealing issues like race conditions or memory leaks
- Supporting better testing and targeted refactoring

Reverse Engineering goals

- Coping with Complexity
- Recover lost information
- Detect Side Effects
- Facilitate Reuse

Coping with Complexity

Goal: Understand and control system complexity by revealing details about architecture, relationships, and design patterns.

Why It Matters: Complex systems are difficult to maintain, evolve, or secure without understanding their internal structure.

Recover lost information

Goal: Retrieve as much information as possible when source code or documentation are lost or unavailable.

What Is Recovered:

- Source code (or higher-level representations)
- Data structures
- Design details
- Functional and architectural knowledge

Detect Side Effects

Goal: Analyze system behavior to identify unintended implications, dependencies, and interactions not apparent from documentation.

Why It Matters: Systems often have behaviors that were never documented—understanding these prevents errors during maintenance.

Facilitate Reuse

- **Goal:** Identify reusable parts or modules in existing systems to improve efficiency and decrease development time.

  Application: By understanding functionality and architecture, developers can extract and repurpose components for new projects